# EXPERIMENT 23

# Deploy a Multi-Container Application Using Kubernetes

## 1. Aim

To deploy a multi-container application consisting of frontend, backend, and database components using Kubernetes, monitor the deployment, and scale the frontend when required.

## 2. Requirements

Docker, Kubernetes, kubectl, Kubernetes Dashboard, frontend, backend, and MongoDB.

## 3. Kubernetes YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nexus-frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nexus-frontend
  template:
    metadata:
      labels:
        app: nexus-frontend
    spec:
      containers:
      - name: frontend
        image: <dockerhub-username>/nexus-frontend:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  selector:
    app: nexus-frontend
  ports:
  - port: 80
    targetPort: 80
```

## 4. Deploy Using kubectl

```
kubectl apply -f deployment.yaml
kubectl get pods
kubectl get services
```

## 5. Monitor and Scale

```
kubectl get pods -o wide
kubectl get svc
kubectl scale deployment nexus-frontend --replicas=3
kubectl get pods
```

## 6. Application Components

Frontend: nexus-frontend    Backend: k8s-backend    Database: mongo-db

## 7. Kubernetes Dashboard – Testing Result

The screenshot below shows the deployed containers running successfully in the Kubernetes dashboard. The frontend, backend, and MongoDB components are shown with active status and exposed ports.

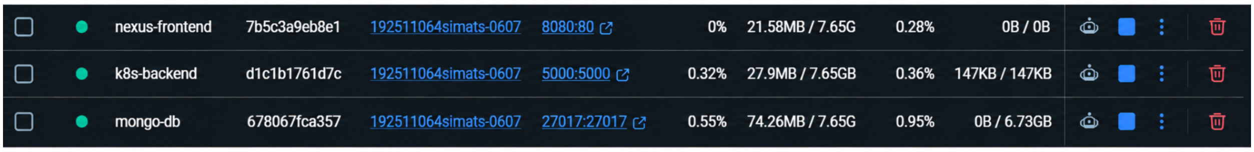


Figure 1: Kubernetes dashboard showing the running multi-container application.

## 8. Result

The multi-container application was successfully deployed using Kubernetes. The frontend, backend, and database containers were running, their status was monitored, and the frontend deployment can be scaled using kubectl.

## 9. Conclusion

Kubernetes simplifies deployment and management of multi-container applications by providing deployments, services, monitoring, and scaling capabilities.